

Nuestras dependencias y por qué no usamos las "oficiales"

Qué es Lightning, por qué mantenemos un fork compatible con Chromium 47 y cómo manejamos los upgrades sin romper la TV.

1 El ecosistema Lightning en 30 segundos

Lightning es un framework open-source para construir apps de TV usando **WebGL** en lugar del DOM tradicional. Tiene dos partes principales:

Renderer

El motor que dibuja en un canvas WebGL (como un game engine para TV).

Solid bindings

El puente que conecta SolidJS con ese renderer.

Lightning lo mantiene **Metrological** (Comcast) y sus paquetes oficiales viven en npm como:

```
@lightningjs/renderer → el motor WebGL
@lightningtv/solid    → los bindings para SolidJS
```

2 Entonces, ¿por qué usamos @solidtv/* ?

Nosotros **no** usamos los paquetes oficiales de Lightning. Usamos un **fork** llamado **SolidTV**:

```
@lightningjs/renderer → @solidtv/renderer (nuestro fork)
@lightningtv/solid    → @solidtv/solid    (nuestro fork)
```

La razón: Samsung Tizen y Chromium 47

Nuestras Samsung TVs corren un navegador interno basado en **Chromium 47** — un browser de **2015**.

Los paquetes oficiales de Lightning (`v3.x`) están hechos para navegadores modernos (Chrome 90+). Si los usáramos directamente, la app **crashearía** en las TVs Samsung porque usan APIs de JavaScript y WebGL que simplemente **no existen** en Chromium 47.

El fork `@solidtv/*` es una versión adaptada que:

- Mantiene compatibilidad con Chromium 47
- Evita APIs modernas que romperían en TVs viejas
- Tiene su propio ciclo de releases independiente

3 Las versiones no son comparables

Esto es importante: ver `v1.x` en nuestro proyecto y `v3.x` en Lightning **no significa que estemos atrasados**. Son proyectos con versionado independiente.

Lo que usamos (fork)	Lo que usa un proyecto Lightning normal
<code>@solidtv/renderer 1.x</code>	<code>@lightningjs/renderer 3.x</code>
<code>@solidtv/solid 1.x</code>	<code>@lightningtv/solid 3.x</code>

Es como comparar la versión de Android de un Samsung con la de un Pixel — mismo origen, distinto release.

4 Por qué las versiones están "clavadas" (sin `^`)

Proyecto web normal	Nosotros
<pre>"@lightningjs/renderer": "^3.6.0"</pre>	<pre>"@solidtv/renderer": "1.6.4"</pre>
Acepta updates automáticos	Versión exacta, sin caret

Tres razones

- 1 Un minor bump puede romper la TV** — una actualización que funciona perfecto en tu laptop puede usar una API que Chromium 47 no tiene. No hay manera de saberlo sin probar en el dispositivo físico.
- 2 Renderer + Solid van de la mano** — si actualizás uno sin el otro, puede haber incompatibilidades internas que solo se manifiestan como crashes en la TV, no como errores de compilación.
- 3 Builds reproducibles** — todos en el equipo y en CI tienen exactamente las mismas dependencias. En un dispositivo donde debuggear es difícil (no hay DevTools como en web), esto elimina variables.

5 Cómo se maneja un upgrade

No es " `yarn upgrade` y listo". El proceso es:

- 1 Revisar changelogs de `@solidtv/renderer` y `@solidtv/solid`
- 2 Subir **ambos** paquetes juntos
- 3 `yarn build:tvsolid` ¿compila?
- 4 `yarn test:tvsolid` ¿pasan los tests?
- 5 **Probar en Samsung Tizen real** ¿funciona en la TV?
- 6 `yarn check:platform-parity` ¿todas las plataformas ok?

El **paso 5 es el más crítico**. Tu laptop usa un Chrome moderno — **no es representativo** del runtime real. Algo puede compilar, pasar tests, verse bien en localhost, y aun así fallar en la TV.

6 Resumen para llevar

Pregunta	Respuesta
¿Por qué no usamos <code>@lightningjs/*</code> ?	Porque apunta a browsers modernos y nuestras TVs corren Chromium 47.
¿Qué es <code>@solidtv/*</code> ?	Un fork de Lightning compatible con Chromium 47.
¿Estamos atrasados en versiones?	No — son líneas de versión diferentes, no comparables.
¿Por qué no ponemos <code>^</code> en las versiones?	Porque un auto-update puede romper la TV sin que lo sepamos.
¿Cómo actualizamos?	Manualmente, los dos paquetes juntos, con prueba en dispositivo físico.
¿Quién mantiene el fork?	chiefcll (Chris Lorenzo) en github.com/solid-tv

7 Links útiles

Fork renderer	github.com/solid-tv/renderer
Fork solid	github.com/solid-tv/solid
Upstream renderer	github.com/lightning-js/renderer
Upstream solid	github.com/lightning-tv/solid
SolidTV docs	solid-tv.github.io/solid/#/essentials/ai_context